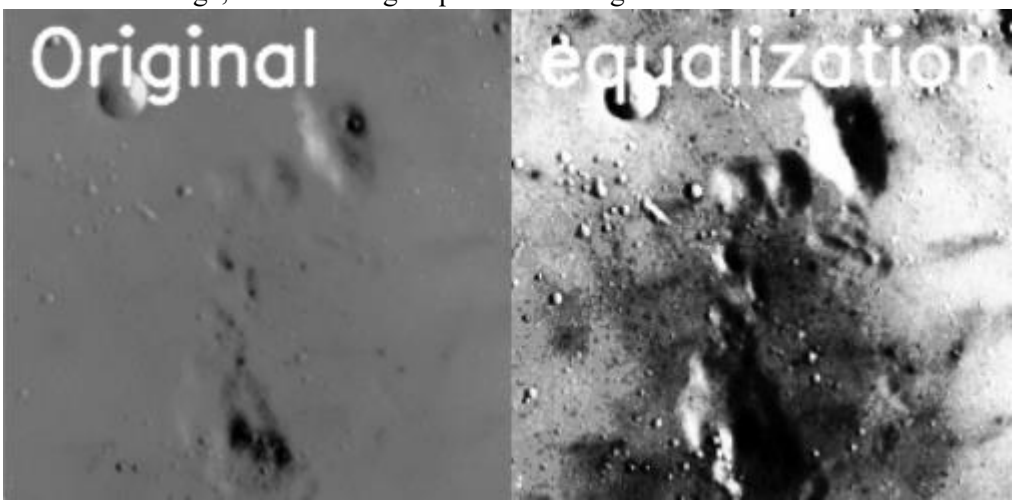
Q1:

a. First and foremost, I defined 3 functions in which they modify the images using various image enhancement methods (Brightness and contrast stretching, Gamma correction, Histogram equalization).
Then proceeded to define another method that takes two images then plots the images one beside the other, which will help me later decide which enhancement is the best for the given image.

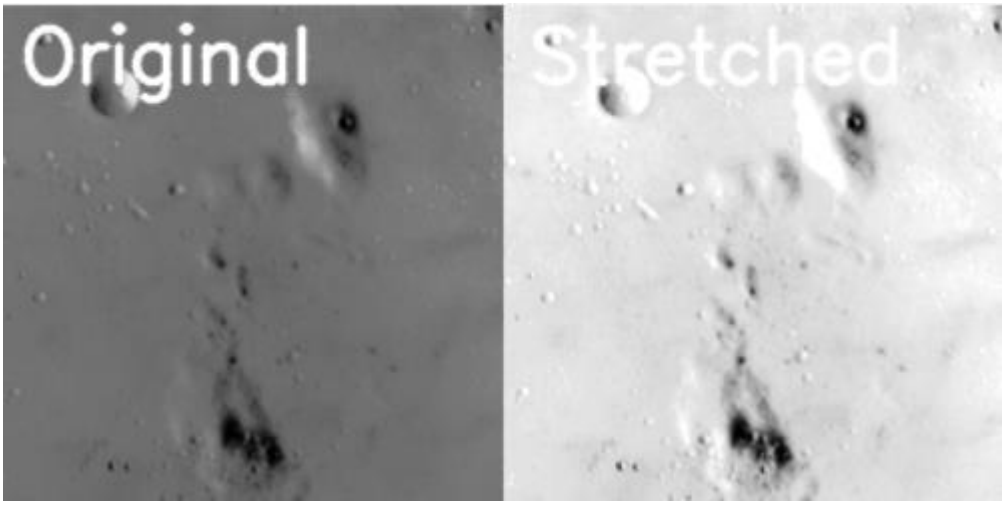
For the first image, if we do image equalization we get:

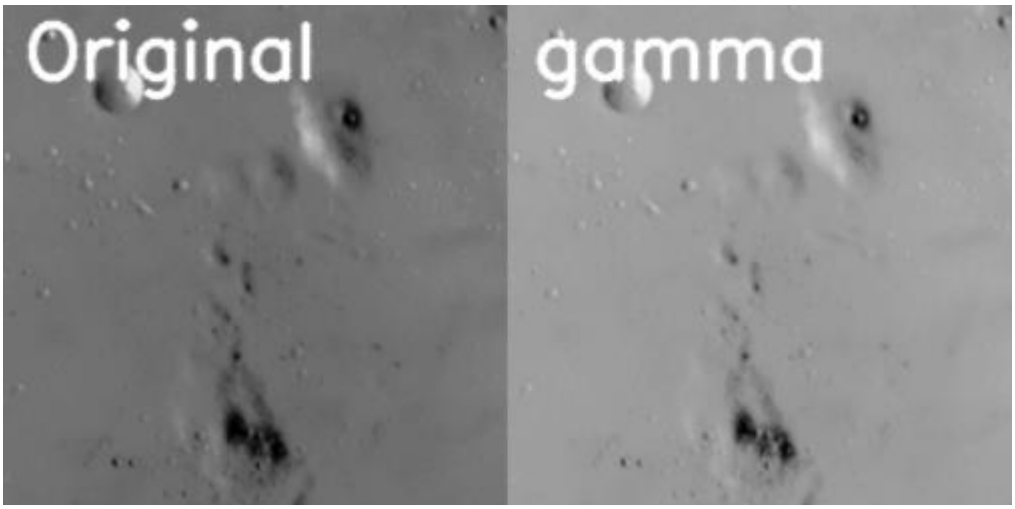


Which is the best for the given image. The main reason is that the original image has tons of low contrast local areas. For this reason, the stretching and gamma correction techniques are less effective since they doesn't contribute much to the situation making it terrible choices compared to equalization technique, since it mainly redistributes the intensities to span a wider range making it uniformly distributed across the image.
Stretching/Gamma correction failing comes from the fact that for pixels with the same intensity, they'll be mapped to the same intensity after the stretching/gamma correction unlike the equalization. We have a lot of same-intensity pixels in the original image meaning they'll do poorly (in other words, still low contrast).

Worth noting that in our case my claim holds for every parameter value's (α, β, γ).

here is an example:





For the second image, if we do gamma correction with $\gamma = 1.2$ we get:



Which is the best for the given image. I think value 1.2 is good for our case because the image is dark and by choosing $\gamma > 1$ the image will be brighter and by setting it to be 1.2 the image will have less pixilated areas, take for example the case when $\gamma = 1.5$:



As we can see, the image is pixelated in the blue circle.

The reason why gamma correction suits our case the best is that, as we can see, there're shadows and highlights in the original image, thus we need to enhance these areas and maintain the smooth transition in the image. Here, the scaling/equalization fail, since the equalization doesn’t preserve the smooth transition and pixilate the image. And the scaling fails in enhancing the shadows (since we need a non-linear enhancement to do so).

Not to forget to mention that gamma correction avoids (in our case) the over-enhancement unlike the equalization.

For example:





For the third image, I think that when $\gamma = 0.9$ the image will be the best represented.



The reason why gamma correction suits our case the best is that, as we can see, there're shadows and highlights in the original image, thus we need to enhance these areas and maintain the smooth transition in the image. We chose a gamma value that's less than 1 because the image is so bright and we thought making it darker would suit it better.

Here, the scaling/equalization fail, since the equalization doesn't preserve the smooth transition and pixilate the image. And the scaling fails in enhancing the shadows (since we need a non-linear enhancement to do so).

Not to forget to mention that gamma correction avoids (in our case) the over-enhancement unlike the equalization.

For example:



This for stretching:



Worth noting that I tried plenty of parameters (α, β, γ) and provided (in the code and the PDF) the ones that gave the best result even though it was terrible.

b. In the code, I defined the functions that do the enhancement techniques and applied them on each image and concluded the results that I just mentioned above for the best choice for each image.
In the apply_fix function, I checked the id of an image and for each image, I saved the proper result according to what we got earlier in fixed_image file (which we'll attach to the final submission).

Q2:

a. In this section, we tried to pick the critical points (such as point on the edges/corners), as well as trying to choose points from different sides of the image (e.g. top, bottom, left and right) and far away from each other as possible and avoid the locations that are close to each other.
The process we used was running the following code and changing the images we're matching accordingly to determine the location of matched points:

```
img1_path = 'puzzles/puzzle_homography_1/pieces/piece_1.jpg'
img2_path = 'puzzles/puzzle_homography_1/pieces/piece_3.jpg'
# Create a figure with two subplots side by side

img1 = mpimg.imread(img1_path)  # Replace with your first image file path
img2 = mpimg.imread(img2_path)  # Replace with your second image file path

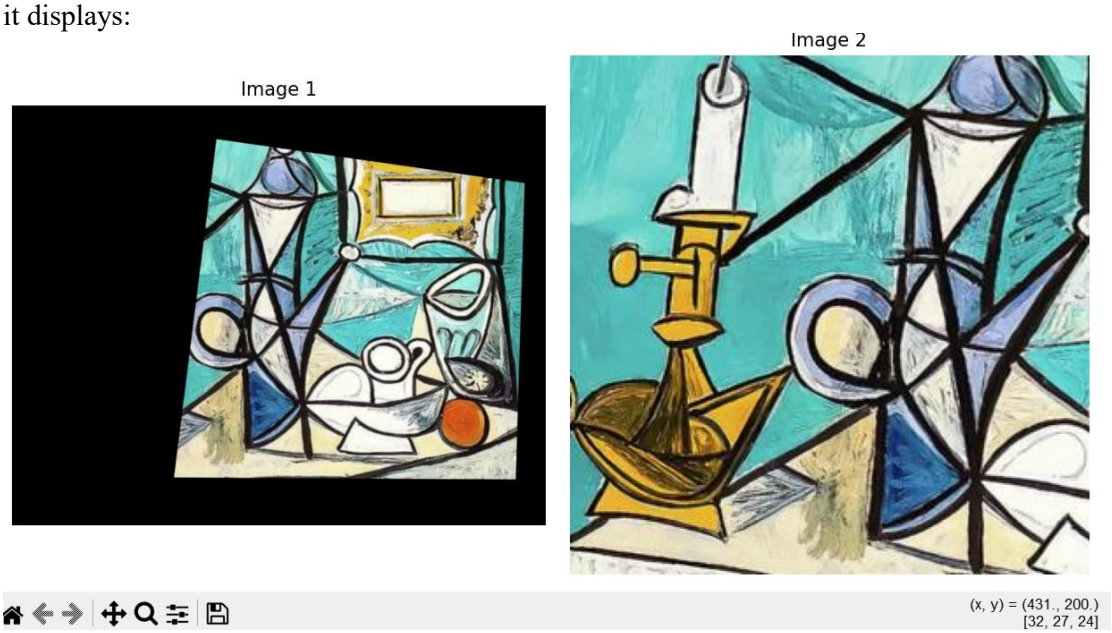
fig, axes = plt.subplots( nrows: 1,  ncols: 2, figsize=(10, 5))  # 1 row, 2 columns

# Plot the first image in the first subplot
axes[0].imshow(img1)
axes[0].axis('off')  # Turn off the axis for better visualization
axes[0].set_title("Image 1")  # Optional title for the first image

# Plot the second image in the second subplot
axes[1].imshow(img2)
axes[1].axis('off')  # Turn off the axis for better visualization
axes[1].set_title("Image 2")  # Optional title for the second image

# Display the images
plt.tight_layout()  # Adjust spacing between subplots
plt.show()
```

In the example above, we're comparing pieces 1 and 3 in puzzile_homography folder, it displays:



And as you can see, in the bottom right corner, the location is shown. In the affine files we chose 3 pairs of matching locations, and in the homography file we chose 4 pairs.

b. We called the function in the following code:

```
# step 3
matches, is_affine, n_images = prepare_puzzle(puzzle)
```

c. In this section, we used a built-in method in the cv2 library to calculate the transform matrix efficiently and to avoid errors that we might get if we implemented it ourselves.
So, the methods we used were:

```
affine_transform_matrix = cv2.getAffineTransform(src_locations, target_locations)
```

For affine transformation.

```
pers_transform_matrix = cv2.getPerspectiveTransform(src_locations, target_locations)
```

For perspective transformation.
With the returned matrix we can make the first piece to "look like" the other original pieces. So, we came up with the idea of switching the locations in the function and keeping the non-black colors in the image to make the original piece look like the first (but we didn't and continued with the original plan).

d. In this function, we'll inverse the transformation matrix we got from the previous section to make the rest of the image to look like the first image. Thus, we started by checking whether the transformation matrix represents affine transformation (by checking the diminutions of the array, since if it's 2×3 matrix, it'll be affine. If it's 3×3 matrix it'll be perspective).
If the matrix is affine then we'll get the proper inverse matrix using:

```
cv2.warpAffine(target_img, original_transform, output_size, flags=cv2.WARP_INVERSE_MAP)
```

If it's perspective:

```
cv2.warpPerspective(target_img, original_transform, output_size, flags=cv2.WARP_INVERSE_MAP)
```

e. We solved it in 2 ways: the classic for loop, and the mask way.

The mask way:

The purpose of this section is to merge/overlay the second image on the first image. To do so, we'll run over the second image and for each pixel that's not black, we'll copy it in the corresponding location in the first image. Notice that the images must be the same size (to be more precise, the first image should at least contain the second image). Hence, we defined a mask of the second image (which is a 2D array of dimensions the same as the second image) where in:

$$mask_{i,j} = true \Leftrightarrow img_2(i,j) \neq 0$$

Meaning in the location that's not black the mask value is true.
Since the mask is 2D and the images are RGB (3D) we'll add a dimension to the mask making it 3D.
The final step is to run over the mask and copy the colors which are true values to image 1.

The classic way:

Run over the image 2 and copy the color of non black locations to image 1.

f. This step is straightforward, it appears directly in the code with the proper explanation.